

Módulo 8 – Capítulo 02 – Sección 01

Por qué cuantizar: reducción de memoria y aceleración de inferencia con pérdida controlada

El modelo base seleccionado en el capítulo anterior pesa, en su forma de entrenamiento en BF16 (2 bytes por parámetro), 14 GB para un modelo de 7B parámetros. Esta cifra define el hardware mínimo requerido sin cuantización: una GPU de 16 GB de VRAM con margen suficiente para buffers de activación y el KV cache. Para la mayoría de los equipos que trabajan con modelos locales, este requisito elimina del presupuesto las GPUs de consumo más accesibles y obliga a optar por hardware de datacenter o nube para inferencia en producción. La cuantización es la técnica que rompe esta barrera: reduce la representación numérica de los pesos de 16 a 4 u 8 bits, comprimiendo el mismo modelo de 7B a 3.5-7 GB con una pérdida de calidad que, para la mayoría de las tareas de producción, es prácticamente imperceptible.

Cuantizar significa representar los pesos del modelo con menos bits que los utilizados durante el entrenamiento. El entrenamiento moderno usa BF16 (bfloat16, 16 bits, con amplio rango dinámico), pero una vez que los pesos están fijados, es posible reducirlos a INT8 (1 byte), INT4 (medio byte efectivo con agrupación) o incluso INT2, asumiendo una pérdida de precisión que impacta en el rendimiento del modelo de formas medibles pero frecuentemente tolerables. La reducción de precisión no es un daño uniforme: algunas capas son mucho más sensibles a la cuantización que otras. Las primeras y últimas capas del modelo (embedding y lm_head) y las capas de atención tienden a ser más críticas; las capas intermedias de las proyecciones FFN toleran mayor compresión. Esta observación es la base de las técnicas de cuantización mixta que aplican más bits donde el modelo es más sensible.

El trade-off cuantitativo de una cuantización Q4_K_M sobre Llama 3 8B ilustra bien la práctica: reducción del 75% en uso de memoria (de 14 GB a ~4.1 GB), velocidad de generación en CPU que puede mejorar respecto a FP16 por menor ancho de banda de memoria necesario, y una degradación de perplexity de aproximadamente 0.15-0.3 puntos en WikiText-2. La perplexity es una métrica de fluencia del modelo (cuánto "sorprenden" al modelo los tokens del texto de test): un incremento de 0.15-0.3 puntos sobre una perplexity base de 5-7

representa una degradación relativa del 2-5%, que para la mayoría de las aplicaciones conversacionales, de extracción de información o de código no es detectable por usuarios humanos.

Esta sección también establece la referencia canónica de la fórmula de memoria del modelo, que será utilizada en los capítulos de hardware (Capítulo 4) y serving (Capítulo 5):

Fórmula de VRAM para pesos del modelo:

$$\text{VRAM_pesos (GB)} = \text{parámetros} \times \text{bits_cuantización} / 8 / 1e9$$

Ejemplos: 7B en Q4: $7e9 \times 4 / 8 / 1e9 = 3.5 \text{ GB}$; en Q8: 7 GB ; en BF16: 14 GB .

A esta cifra debe sumarse el **KV cache** (que se detalla en el Capítulo 4) y un **overhead del framework** del 10-15% para buffers de activación y el runtime de CUDA o Metal. En hardware con VRAM limitada, la cuantización es la palanca que hace posible cargar el modelo; el KV cache y el overhead determinan si queda VRAM suficiente para el contexto de inferencia.

Aspectos técnicos de la cuantización

- **Cuantización post-entrenamiento (PTQ):** se aplica sobre pesos ya entrenados sin reentrenamiento; la técnica más usada en producción; incluye GPTQ, AWQ y GGUF con variantes K-quant.
- **Cuantización durante el entrenamiento (QAT):** simula el error de cuantización durante el fine-tuning; produce modelos más robustos ante cuantización pero requiere el pipeline de entrenamiento completo.
- **Tipos de datos:** FP32 (4 bytes), BF16 (2 bytes, rango dinámico amplio, preferido para entrenamiento), FP16 (2 bytes, riesgo de overflow en rangos extremos), INT8 (1 byte), INT4 (0.5 bytes efectivos con agrupación).
- **Cuantización por grupos:** factores de escala separados por grupos de 32 o 128 pesos mejoran la precisión a costa de ~3-5% de overhead de memoria adicional.
- **Aceleración en hardware:** las GPUs NVIDIA con Tensor Cores aceleran operaciones INT8 e INT4 mediante instrucciones especializadas (DP4A, IMMA); Apple Silicon acelera INT8 en el Neural Engine; CPUs con AVX-512 VNNI aceleran INT8 nativo.

Nota del Arquitecto: El proceso correcto de cuantización no es "comprimir al máximo posible" sino "encontrar el mínimo de bits que no degrada la calidad en la tarea específica del producto". Evalúa siempre Q4_K_M primero en tu golden dataset: si la calidad es suficiente, ganas el 75% de reducción de memoria. Si no es suficiente, prueba Q5_K_M. Rara vez necesitarás más de Q6_K para aplicaciones de producción que no sean matemáticas avanzadas o generación de código crítico.

La cuantización es la primera técnica de ingeniería aplicada al modelo seleccionado, y determina el hardware mínimo viable para el despliegue. Las secciones siguientes detallan los tres formatos principales de cuantización para producción: GGUF para inferencia CPU/GPU con llama.cpp, GPTQ para máximo throughput en GPU NVIDIA, y AWQ para la mejor relación calidad/compresión en modelos pequeños.

Módulo 8 – Capítulo 02 – Sección 02

GGUF: formato para inferencia en CPU y GPU con llama.cpp

El trabajo de construir y distribuir modelos cuantizados requiere un formato estándar que encapsule todo lo necesario para la inferencia en un único archivo portátil: los pesos cuantizados, la configuración de arquitectura del modelo, el tokenizador y los metadatos de cuantización. Antes de GGUF, los ingenieros que querían ejecutar modelos localmente necesitaban gestionar múltiples archivos de configuración separados, aplicar parches de cuantización manualmente y lidiar con incompatibilidades entre versiones de librerías. GGUF resolvió este problema de distribución con un diseño de archivo único autocontenido que funciona en cualquier plataforma donde compile llama.cpp.

GGUF (GPT-Generated Unified Format) es el formato de archivo binario diseñado por Georgi Gerganov como sucesor de GGML. Su característica más importante desde el punto de vista operativo es que el archivo es completamente **autocontenido y multiplataforma**: contiene los pesos cuantizados, la configuración del modelo (número de capas, cabezas de atención, dimensión oculta, frecuencia base de RoPE), el vocabulario y los templates de chat, y los factores de escala de cuantización por grupo. Para el AI Engineer, esto significa que copiar un archivo `.gguf` de una máquina a otra es suficiente para ejecutar el modelo en el destino sin instalación adicional de modelos o configuraciones externas.

La portabilidad multiplataforma es consecuencia directa del diseño de GGUF: llama.cpp compila con soporte para Metal (Apple Silicon), CUDA (NVIDIA), ROCm (AMD), Vulkan (cross-platform) y CPU pura con instrucciones AVX2/AVX-512. El mismo archivo `.gguf` ejecuta sin modificación en todas estas plataformas; lo que cambia entre plataformas es el backend de aceleración que llama.cpp utiliza en tiempo de ejecución.

La nomenclatura estándar de los archivos GGUF codifica la variante de cuantización directamente en el nombre. Un archivo `llama-3-8b-instruct.Q4_K_M.gguf` indica cuantización K-quant de 4 bits con mezcla media (M). Esta convención de nomenclatura permite identificar inmediatamente el nivel de cuantización sin necesitar abrir el archivo ni leer metadatos. Las variantes K-quant disponibles son: Q2_K, Q3_K (en variantes S/M/L),

Q4_K (S/M), Q5_K (S/M), Q6_K y Q8_o, donde el número indica los bits por peso predominantes y la letra final el tamaño del grupo de cuantización utilizado (S=small, M=medium, L=large); grupos mayores ofrecen mejor calidad al costo de mayor overhead de memoria para los factores de escala.

Una característica operativamente relevante de GGUF es el soporte de **carga parcial en GPU mediante offloading por capas**. Si el modelo no cabe completamente en la VRAM disponible, llama.cpp puede cargar las primeras N capas en GPU (donde ocurre la mayor parte del cómputo intensivo) y las capas restantes en la RAM del sistema, ejecutándolas en CPU. Esto se controla con el flag `--n-gpu-layers N` y permite hacer viable la inferencia de modelos que exceden la VRAM disponible, aunque con menor velocidad que la carga completa en GPU. La selección del número óptimo de capas a cargar en GPU es un problema de optimización empírico que depende del tamaño del modelo, la cuantización y las memorias disponibles.

Componentes del formato GGUF

- **Estructura del archivo:** cabecera de identificación seguida de metadatos key-value (configuración del modelo, tokenizador, template de chat, información de cuantización) y los tensores cuantizados almacenados en orden de dependencia de inferencia.
- **Metadatos embebidos:** incluye toda la información necesaria para cargar y usar el modelo: número de capas, cabezas de atención, dimensión oculta, tipo de RoPE, vocabulario completo, tokens especiales y template de chat en formato Jinja2.
- **Variantes K-quant:** Q2_K, Q3_K_S/M/L, Q4_K_S/M, Q5_K_S/M, Q6_K; variantes con más bits son de mayor calidad pero mayor tamaño.
- **Offloading parcial:** el flag `--n-gpu-layers N` de llama.cpp permite cargar las N primeras capas en VRAM y las restantes en RAM del sistema; balance dinámico entre velocidad y disponibilidad de memoria.
- **Compatibilidad multiplataforma:** el mismo archivo funciona con backends Metal, CUDA, ROCm, Vulkan y CPU sin recompilar el modelo.

Nota del Arquitecto: Cuando descargues un modelo GGUF de Hugging Face, el repositorio suele ofrecer múltiples variantes de cuantización del mismo modelo. Si el hardware tiene 8 GB de VRAM, Q4_K_M es casi siempre el punto de inicio correcto. Si tienes 12-16 GB, considera Q5_K_M para mejor calidad. La variante Q8_o es útil cuando necesitas la mayor calidad posible sin recurrir a BF16 completo,

especialmente para tareas de razonamiento matemático o generación de código crítico.

GGUF es el formato que hace que llama.cpp y Ollama sean posibles como herramientas de distribución de modelos: sin un formato binario único y autocontenido, la experiencia de "descargar y ejecutar" un modelo local sería tan compleja como configurar PyTorch, CUDA y los modelos por separado. La sección siguiente examina GPTQ, el formato alternativo optimizado para el máximo throughput en GPU NVIDIA cuando el tiempo de inferencia es la métrica crítica.

Módulo 8 – Capítulo 02 – Sección 03

GPTQ: cuantización post-entrenamiento de 4 bits optimizada para GPU

El formato GGUF con K-quants es la solución universal para inferencia local en cualquier plataforma, pero no es el formato de mayor throughput en GPU NVIDIA cuando la velocidad de generación de tokens es la métrica crítica. GPTQ fue diseñado con un objetivo diferente: extraer el máximo rendimiento de inferencia en GPU aplicando un algoritmo matemáticamente sofisticado que minimiza el error de cuantización de forma mucho más inteligente que el redondeo directo. El resultado es un modelo en INT4 que se comporta en GPU NVIDIA con throughput superior al de las variantes K-quant del mismo nivel de compresión.

GPTQ (Generative Pre-trained Transformer Quantization), desarrollado por Frantar et al. en 2022, resuelve el problema de cuantización capa por capa usando el Optimal Brain Surgeon (OBS) framework: cuantiza los pesos de una fila de la matriz de pesos en orden óptimo (determinado por la matriz Hessiana del error), ajustando los pesos restantes no cuantizados de esa misma fila para compensar el error introducido por cada cuantización individual. Este proceso es fundamentalmente más costoso computacionalmente que el redondeo directo —y por eso tarda horas en lugar de minutos— pero produce modelos con calidad significativamente superior para el mismo nivel de compresión.

La novedad de GPTQ respecto a técnicas anteriores es que adapta el OBS framework para escalar a modelos de decenas de miles de millones de parámetros. Para lograrlo, trabaja en bloques (columnas de la matriz de pesos) y usa una versión aproximada de la Hessiana calculada sobre un dataset de calibración: entre 128 y 1024 muestras de texto representativas del dominio de uso. La elección del dataset de calibración es una decisión de ingeniería con impacto en la calidad: calibrar con datos del mismo dominio del producto mejora la calidad del modelo cuantizado para ese dominio específico respecto a calibrar con datos genéricos de C4 o WikiText. Para un modelo de código, calibrar con código Python y SQL puede mejorar el rendimiento de cuantización en esas tareas.

Los modelos GPTQ se distribuyen en formato SafeTensors en Hugging Face con sufijos `-GPTQ` o `-4bit` en el nombre del repositorio, e incluyen un archivo `quantize_config.json` con los parámetros de cuantización usados: número de bits, tamaño de grupo (`group_size`, típicamente 128), y si la cuantización de activaciones está habilitada. Para la inferencia, los modelos GPTQ requieren librerías específicas: `auto-gptq` es la opción más compatible con el ecosistema Hugging Face y permite cargar el modelo con `AutoGPTQForCausalLM.from_quantized(model_name)`. Para mayor velocidad, `exllamav2` ofrece kernels GPU optimizados que pueden superar el throughput de `auto-gptq` en un factor de 2-3x en GPUs NVIDIA modernas como la RTX 4090.

GPTQ con 4 bits y `group_size=128` logra perplexity comparable a FP16 en la mayoría de los benchmarks, con una degradación inferior al 1% en WikiText-2 para modelos de 7B o más parámetros. En comparación, la variante Q4_K_M de GGUF logra una degradación ligeramente superior porque su algoritmo de cuantización es más simple. Para producción en GPU NVIDIA donde el throughput de tokens por segundo es la métrica crítica, GPTQ con `exllamav2` es la opción preferida; para portabilidad y compatibilidad con múltiples plataformas, GGUF sigue siendo la elección correcta.

Aspectos técnicos de GPTQ

- **Proceso de calibración:** 128-1024 muestras de texto para calcular la Hessiana; duración de 30 minutos a varias horas según el tamaño del modelo y la GPU; calibrar con datos del dominio del producto mejora la calidad para ese dominio.
- **Tamaño de grupo:** `group_size` de 32, 64 o 128 (128 es el estándar más usado); grupos más pequeños mejoran la calidad pero aumentan el overhead de los factores de escala.
- **Formato de distribución:** SafeTensors con `quantize_config.json` en el repositorio de Hugging Face; identificables por el sufijo `-GPTQ` o `-4bit`.
- **Librerías de inferencia:** `auto-gptq` para compatibilidad máxima; `exllamav2` para 2-3x mayor throughput en GPUs NVIDIA modernas con kernels CUDA optimizados.
- **Compatibilidad de hardware:** requiere soporte para dequantización INT4 eficiente en GPU; funciona desde GPUs Pascal (GTX 1080) en adelante para NVIDIA; soporte AMD ROCm limitado en algunas librerías.

Nota del Arquitecto: El tiempo de calibración de GPTQ es una inversión única: una vez cuantizado, el modelo puede distribuirse y reutilizarse. En producción, la decisión entre GPTQ y GGUF Q4_K_M se reduce a: si usas exclusivamente GPU

NVIDIA y la librería exllamav2 es viable en tu stack, GPTQ da mayor throughput; si necesitas portabilidad multiplataforma (Mac, AMD, CPU) o integración con Ollama, GGUF es más simple. Para la mayoría de los equipos pequeños, GGUF es la elección pragmática correcta porque reduce la complejidad operativa.

La siguiente sección examina AWQ, una técnica de cuantización más reciente que logra mejor calidad que GPTQ en modelos pequeños con un proceso de cuantización mucho más rápido, particularmente relevante para equipos que necesitan iterar entre versiones de modelos frecuentemente.

Módulo 8 – Capítulo 02 – Sección 04

AWQ (Activation-aware Weight Quantization): menor pérdida de calidad que GPTQ

Tanto GPTQ como los K-quants de GGUF tratan los pesos del modelo de forma relativamente uniforme, aplicando el mismo proceso de cuantización a todos los canales de una capa (con diferencias en sofisticación del algoritmo, pero no en la información usada para decidir qué cuantizar agresivamente y qué proteger). AWQ parte de una observación diferente y más poderosa: los pesos no son uniformemente importantes para la calidad del modelo, y los pesos que afectan más al output son precisamente los que corresponden a las activaciones de mayor magnitud en el dataset de calibración. Si se pueden identificar y proteger estos pesos críticos, el resto puede cuantizarse más agresivamente con menor impacto total en la calidad.

AWQ (Activation-aware Weight Quantization), desarrollada por Lin et al. del MIT-Han Lab en 2023, implementa esta intuición de forma matemáticamente elegante. El proceso de cuantización analiza las estadísticas de activación en un dataset de calibración (128 a 512 muestras son suficientes) e identifica el 1% de los canales de salida con activaciones de mayor magnitud media. En lugar de proteger directamente estos canales aplicando más bits —lo que rompería la uniformidad del formato de cuantización y complicaría la implementación de kernels eficientes— AWQ aplica un scaling matemático previo: escala los pesos importantes en un factor $s > 1$ y las activaciones correspondientes en $1/s$, preservando la equivalencia matemática de la capa pero haciendo que la cuantización introduzca un error relativo menor en los canales importantes. El resultado es un modelo INT4 con calidad consistentemente superior a GPTQ para el mismo nivel de compresión, especialmente pronunciado en modelos pequeños de 1B-7B donde la cuantización agresiva tiene mayor impacto relativo.

La ventaja operativa más inmediata de AWQ respecto a GPTQ es la velocidad de cuantización: el proceso completo para un modelo de 7B tarda minutos en lugar de horas porque no resuelve sistemas de ecuaciones de segunda orden. Esto hace que AWQ sea especialmente relevante para equipos que iteran frecuentemente entre versiones de modelos fine-tuneados y necesitan re-cuantizar en cada iteración del ciclo de desarrollo. La integración con el

ecosistema de producción es también directa: el paquete `autoawq` genera modelos compatibles con vLLM directamente mediante:

```
from awq import AutoAWQForCausalLM

model = AutoAWQForCausalLM.from_pretrained("meta-llama/Llama-3.1-8B-Instruct")

model.quantize(tokenizer, quant_config={"zero_point": True, "q_group_size": 128, "w_bit": 4})

model.save_quantized("./Llama-3.1-8B-Instruct-AWQ")
```

Los modelos resultantes se cargan en vLLM con `--quantization awq` sin configuración adicional, permitiendo usar el mismo pipeline de serving para modelos en FP16, GPTQ y AWQ con un único flag de diferencia.

En benchmarks comparativos sobre Llama 2-7B, AWQ-4bit logra perplexity de 5.78 en WikiText-2 versus 5.84 de GPTQ-4bit y 5.47 del modelo FP16 original. Esta diferencia de 0.06 puntos representa una degradación relativa aproximadamente 40% menor que GPTQ. Para modelos más grandes (13B o más), la diferencia entre AWQ y GPTQ se reduce porque ambos métodos tienen más "presupuesto" de bits para mantener la calidad. Los modelos AWQ se distribuyen en Hugging Face con el sufijo `-AWQ` en el nombre del repositorio, incluyen un archivo `quant_config.json` y son compatibles con carga directa en `transformers` desde la versión 4.35.

Componentes principales de AWQ

- **Búsqueda de canales importantes:** análisis de estadísticas de activación en 128-512 muestras; identifica el 1% de canales con mayor magnitud media de activación; proceso en minutos, no horas.
- **Scaling pre-cuantización:** escala matemática de pesos y activaciones para reducir el error de cuantización en canales importantes sin cambiar el formato de representación.
- **Compatibilidad con vLLM:** modelos AWQ son compatibles de forma nativa con vLLM vía `--quantization awq`; también compatibles con transformers directamente desde la versión 4.35.
- **Formato de distribución:** SafeTensors con `quant_config.json`; identificables por el sufijo `-AWQ` en Hugging Face.
- **Comparación de calidad:** AWQ-4bit logra degradación de perplexity aproximadamente 40% menor que GPTQ-4bit para modelos de 7B; diferencia se reduce

para modelos de 13B o más.

Nota del Arquitecto: AWQ ha desplazado a GPTQ como primera opción de cuantización para GPU en muchos equipos porque combina lo mejor de dos mundos: mejor calidad que GPTQ y proceso de cuantización 10-20x más rápido. Si estás comenzando un proyecto y necesitas cuantización para GPU, AWQ es el punto de inicio recomendado. Si tu stack ya usa GPTQ con exllamav2 y el throughput es el factor dominante, el cambio puede no justificarse sin benchmarking específico.

La elección entre GGUF, GPTQ y AWQ depende del hardware objetivo, la librería de inferencia usada y la frecuencia de iteración sobre modelos. La sección siguiente pone estas alternativas en perspectiva con una comparación directa de las variantes más comunes, incluyendo los números de calidad y velocidad que permiten tomar decisiones informadas para un proyecto específico.

Módulo 8 – Capítulo 02 – Sección 05

Comparación: Q4_K_M vs Q5_K_M vs Q8_0 — trade-offs de precisión y velocidad

Seleccionar la variante de cuantización correcta para un despliegue concreto no es una decisión estética sino un análisis de ingeniería: el modelo cuantizado demasiado agresivamente puede fallar en la tarea objetivo del producto, mientras que el cuantizado insuficientemente ocupa más memoria de la necesaria y limita las opciones de hardware viables. Las variantes K-quant del formato GGUF —Q4_K_M, Q5_K_M, Q6_K y Q8_0— representan cuatro puntos bien definidos en este espacio de trade-offs, cada uno con un perfil de tamaño, velocidad y calidad documentado empíricamente.

La nomenclatura K-quant codifica la estrategia de cuantización: el número indica los bits base, la letra K señala que se aplica cuantización adaptativa con factores de escala por grupos, y la letra final (S, M, L) indica el tamaño de esos grupos. Q4_K_M usa 4 bits para la mayoría de los pesos pero eleva a 6 bits las capas más sensibles del modelo (las primeras y últimas capas, más las cabezas de atención críticas), lo que explica por qué logra 4.65 bits efectivos por parámetro en lugar de exactamente 4. Para un modelo de 7B parámetros, esto produce un archivo de aproximadamente 4.1 GB que se genera entre 10 y 20 tokens por segundo en CPU moderna, con una degradación de perplexity de 0.15-0.25 puntos respecto al modelo en BF16. Esta combinación hace de Q4_K_M el punto de inicio recomendado para la mayoría de los despliegues en hardware de consumo.

Q5_K_M incrementa los bits efectivos a 5.68 por parámetro, resultando en un archivo de aproximadamente 5.0 GB para el mismo modelo de 7B. La mejora de calidad respecto a Q4_K_M es consistente pero moderada: la degradación de perplexity cae a 0.05-0.10 puntos, una mejora que en evaluaciones de usuario humano frecuentemente solo se percibe en tareas con alta densidad de razonamiento o en generación de código con lógica compleja. Para equipos con hardware de 8-10 GB de VRAM o RAM y que reportan degradación visible con Q4_K_M en su tarea específica, Q5_K_M es el siguiente paso natural antes de considerar hardware adicional.

Q6_K alcanza 6.56 bits efectivos y degradación de perplexity inferior a 0.05 puntos: en evaluaciones ciegas por usuarios humanos en tareas conversacionales y de extracción de información, esta diferencia respecto al modelo FP16 original es prácticamente indetectable. Q6_K ocupa 5.9 GB para un modelo de 7B y es la elección correcta para tareas donde la precisión es crítica —generación de código en dominios especializados, síntesis de documentos médicos, razonamiento matemático— sin necesidad de incrementar el hardware al completo modelo en BF16.

Q8_o es cuantización de 8 bits sin agrupamiento, produciendo archivos de 7.2 GB con pérdida de perplexity inferior a 0.01 puntos: es prácticamente indistinguible del modelo original en cualquier benchmark. Su desventaja frente a las K-quants es que no aprovecha la aceleración INT4 de los Tensor Cores NVIDIA y ocupa significativamente más memoria sin la mejora de calidad que justificaría ese tamaño adicional. Q8_o tiene sentido principalmente para uso en CPU cuando se quiere la máxima calidad posible sin modificar el hardware y la velocidad de generación es secundaria.

Trade-offs de precisión y velocidad

- **Q4_K_M (recomendado para uso general):** 4.65 bits efectivos, ~4.1 GB para 7B, pérdida de perplexity de 0.15-0.25 puntos; mejor relación calidad/tamaño; para hardware con 6-8 GB de RAM/VRAM.
- **Q5_K_M (calidad prioritaria):** 5.68 bits efectivos, ~5.0 GB para 7B, pérdida de 0.05-0.10 puntos; cuando la calidad importa más que el tamaño y se dispone de 8-10 GB.
- **Q6_K (casi sin pérdida):** 6.56 bits efectivos, ~5.9 GB para 7B, pérdida inferior a 0.05 puntos; para tareas críticas como código especializado o razonamiento médico.
- **Q8_o (cuantización mínima):** 8 bits, ~7.2 GB para 7B, pérdida inferior a 0.01 puntos; máxima calidad en CPU cuando el tamaño no es restricción.
- **Velocidad en CPU vs GPU:** Q4_K_M es hasta un 30% más rápido que Q8_o en CPU por menor ancho de banda de memoria necesario; en GPU con aceleración INT4, Q4_K_M puede ser 2x más rápido en throughput de tokens.

Nota del Arquitecto: El proceso correcto de selección de variante es empírico, no teórico: descarga el modelo en Q4_K_M, ejecútalo en tu golden dataset del Capítulo 1, y mide si la calidad cumple el umbral del producto. Si no cumple, sube a Q5_K_M y repite. En más del 80% de los proyectos de producción que he visto, Q4_K_M supera

el umbral de calidad y permite usar hardware significativamente más económico que el requerido por Q8_0 o BF16.

La elección de variante de cuantización completa el perfil técnico del modelo seleccionado: combinando familia, tamaño y variante K-quant, el AI Engineer puede calcular exactamente los requisitos de hardware para el despliegue. El siguiente capítulo cubre las herramientas que ejecutan estos modelos GGUF en la práctica, comenzando con llama.cpp y Ollama.

Módulo 8 – Capítulo 02 – Sección 06

Cierre: la cuantización es la técnica que hace posible ejecutar LLM en hardware accesible

La cuantización ha transformado radicalmente el panorama del despliegue de LLMs en los últimos tres años. Lo que en 2022 requería un clúster de GPUs A100 para ser viable puede ejecutarse en 2025 en una laptop con 16 GB de RAM o en una GPU de consumo de 8 GB de VRAM con calidad apenas degradada para la mayoría de las tareas de producción. Esta democratización no es el resultado de mejoras incrementales en hardware sino de avances algorítmicos en representación numérica: GPTQ, AWQ y los K-quants de GGUF han madurado al punto donde la degradación de calidad en 4 bits es frecuentemente indetectable por usuarios finales en tareas conversacionales y de extracción de información.

Las tres técnicas de cuantización presentadas en este capítulo cubren diferentes necesidades operativas con el mismo objetivo. GPTQ optimiza el throughput en GPU NVIDIA con un proceso de calibración que tarda horas pero produce los modelos más eficientes para inferencia en hardware NVIDIA; es la elección para producción de alta demanda donde la velocidad por token es el KPI principal. AWQ prioriza la calidad sobre la velocidad de cuantización, identificando los canales más críticos del modelo y protegiéndolos con mayor precisión; su proceso tarda minutos y produce modelos con menor degradación que GPTQ, especialmente en modelos pequeños de 1B-7B. Las K-quants de GGUF equilibran ambas dimensiones dentro del ecosistema de llama.cpp: compatibles con CPU, GPU y Apple Silicon sin cambios en el archivo del modelo, con variantes que permiten ajustar el trade-off precisión/tamaño entre Q2_K y Q8_o según los requisitos del hardware de despliegue.

El proceso de selección de cuantización ha llegado a ser parte del ciclo de vida estándar del modelo en organizaciones maduras. Un mismo modelo base —por ejemplo, Llama 3.1 8B— puede existir simultáneamente en Q4_K_M para despliegue en laptops de desarrolladores, en AWQ-4bit para el servidor de inferencia de producción con GPU A10G, y en BF16 completo para los runs de fine-tuning con QLoRA. Cada variante sirve un propósito distinto dentro del mismo ecosistema del producto, y la habilidad de gestionar estas variantes como artefactos de

primera clase en el registry de modelos —con sus propias métricas de calidad evaluadas en el golden dataset— es una competencia diferenciadora del AI Engineer moderno.

La cuantización también es el prerrequisito del fine-tuning eficiente que se verá en el Capítulo 6: QLoRA carga el modelo base en NF4 (4-bit NormalFloat) durante el entrenamiento para reducir el footprint de memoria a la mitad respecto a BF16, permitiendo fine-tuning de modelos de 70B en una sola GPU A100 de 40 GB. El NF4 de QLoRA es una variante de cuantización diseñada específicamente para los valores de los pesos de redes neuronales, y su comprensión técnica requiere los conceptos de cuantización por grupos y de tipos de datos especializados introducidos en este capítulo.

Idea central

La cuantización no es una solución de compromiso sino una decisión de ingeniería de sistemas: el modelo cuantizado correcto en el hardware correcto supera en latencia y costo total de propiedad a un modelo sin cuantizar en hardware más caro.

"Quantization is not about losing quality — it's about finding the right precision for the right operation." — Tim Dettmers, investigador de cuantización de redes neuronales y autor de bitsandbytes y QLoRA, sobre el principio fundamental que guía las técnicas modernas de compresión de modelos.